

PROJET : DAILY PLANZ

28/05/2026

Équipe :2

OUISSAL NOURI

HOUDA EL MKKADEM

NADA SARHANE

Scrum Master

Product Owner

Développeur

Contenu du document :

Partie 1 – Rappel du projet

Partie 2 – Product Backlog initial

Partie 3 – Planification des Sprints (Sprint 1 & Sprint 2)

Partie 4 – Simulation des cérémonies Scrum (Sprint 1)

Partie 5 – Definition of Ready (DoR) et Definition of Done

Partie 6 – Suivi et indicateurs (Burndown, Jira, risques Scrum)

Enseignant :

Pr. Sanae EL MIMOUNI

Méthodologies de Développement Logiciel – 2025-2026

Partie 1 — Rappel du projet

Objectif du projet et public cible

DailyPlanz est un site web de productivité dont l'objectif est d'aider l'utilisateur à mieux organiser ses tâches quotidiennes. Il permet de créer, consulter, modifier, supprimer et marquer des tâches comme terminées dans une interface simple et rapide. Le public cible est toute personne souhaitant gérer ses tâches personnelles (étudiants, employés, etc.). Cette version est pensée pour un usage facile et immédiat, sans complexité inutile.

Rappel synthétique (issu du livrable 2)

La durée totale prévue du projet (selon l'estimation du livrable 2) est d'environ 2 mois. Dans ce livrable Scrum, nous avons planifié 2 sprints de 2 semaines pour livrer un MVP fonctionnel. La vélocité initiale est estimée de manière empirique à 16–17 Story Points par sprint (équipe étudiante, pas d'historique de sprints).

Justification de la répartition des rôles Scrum

Nous avons choisi Houda comme Product Owner car elle peut clarifier les besoins, définir les priorités et valider les fonctionnalités livrées.

Nous avons choisi Ouissal comme Scrum Master car elle est la plus disponible pour organiser le travail et animer les réunions Scrum.

Nada occupe le rôle de Développeur car elle se concentre principalement sur l'implémentation technique et les tests.

Partie 2-- Product Backlog

ID	User Story (En tant que..., je veux..., afin de...)	Critères d'acceptation	Story Points	Priorité (MoSCoW)	Sprint cible
US-01	En tant que développeur , je veux initialiser le projet (repo, Spring Boot, MySQL, CI) afin de démarrer le développement proprement .	1) L'application démarre sans erreur. 2) La connexion MySQL fonctionne.	3	Must	Sprint 1
US-02	En tant qu' utilisateur , je veux voir la page "Mes tâches" afin de gérer mes tâches .	1) La page s'affiche correctement. 2) La page contient un champ "Ajouter" + une liste (même vide).	2	Must	Sprint 1
US-03	En tant qu' utilisateur , je veux ajouter une tâche afin de noter ce que je dois faire .	1) Si le titre est vide ⇒ message d'erreur. 2) Si le titre est valide ⇒ la tâche apparaît dans la liste.	5	Must	Sprint 1
US-04	En tant qu' utilisateur , je veux consulter la liste des tâches afin de suivre ce que j'ai à faire .	1) Les tâches enregistrées s'affichent. 2) Si aucune tâche ⇒ "Aucune tâche".	3	Must	Sprint 1
US-05	En tant qu' utilisateur , je veux que mes tâches soient enregistrées en base de données afin de les retrouver après actualisation .	1) Après refresh, les tâches restent. 2) Les tâches affichées proviennent de MySQL.	3	Must	Sprint 1
US-06	En tant qu' utilisateur , je veux modifier une tâche afin de corriger son contenu .	1) La modification est sauvegardée et visible. 2) Titre vide ⇒ modification refusée + message.	3	Should	Sprint 2
US-07	En tant qu' utilisateur , je veux supprimer une tâche afin de retirer ce qui n'est plus utile .	1) Confirmation avant suppression. 2) Après refresh, la tâche ne revient pas.	2	Should	Sprint 2
US-08	En tant qu' utilisateur , je veux marquer une tâche comme terminée afin de suivre mon avancement .	1) Statut "terminée" visible (ex : texte barré). 2) Statut conservé après refresh.	2	Should	Sprint 2

US-09	En tant qu' utilisateur , je veux des messages d'erreur clairs afin de comprendre pourquoi une action échoue .	1) Erreurs de formulaire affichées côté UI. 2) Erreur serveur ⇒ message simple, sans crash.	2	Should	Sprint 2
US-10	En tant que développeur , je veux ajouter des tests + CI afin de réduire les bugs .	1) Les tests unitaires passent en local. 2) Le pipeline CI est "green".	3	Should	Sprint 2
US-11	En tant qu' administrateur système , je veux déployer l'application en recette afin de tester avant la livraison .	1) Une URL de recette est accessible. 2) Ajouter/afficher/modifier/supprimer fonctionnent en recette.	5	Must	Sprint 2

Partie 3 — Planification des Sprints

3.1 Vue d'ensemble des sprints (Release planning)

Sprint	Période Sem. 1 – Sem.	Sprint Goal	US incluses US-01, US-02,	Total pts
Sprint 1	2 Sem. 3 – Sem.	Livrer un MVP : page "Mes tâches" + ajout + affichage + sauvegarde BD	US-03, US-04, US-05 US-06, US-07,	16
Sprint 2	4	Compléter la gestion : modifier/supprimer/terminée + qualité + déploiement recette	US-08, US-09, US-10, US-11	17

3.2 Sprint Backlog détaillé — Sprint 1

US	Sous-tâche technique	Responsable	Estimation (h)	Statut
US-01	Créer le repo GitHub + structure (frontend/backend)	Ouissal	2h	À faire
US-01	Initialiser Spring Boot (projet, packages, config)	Nada	3h	À faire
US-01	Configurer MySQL + app lic at ion .pr op ert ie s	Nada	2h	À faire
US-01	tas k	Nada	2h	À faire
US-01	Mettre en place CI (GitHub Actions build/test)	Ouissal	3h	À faire
US-01	Rédiger README Créer schéma BD (table installation + lancement)	Houda	2h	À faire
US-02	Maquette simple UI "Mes tâches" (HTML/CSS)	Houda	4h	À faire

US-02	Intégrer JS de base (structure, événements)	Houda	2h	À faire
US-03	Créer Entity/DTO Task (title, done, dates)	Nada	2h	À faire
US-03	Implémenter API POST /tasks + validation	Nada	4h	À faire
US-03	UI ajout tâche + appel API (fetch/axios)	Houda	3h	À faire
US-04	Implémenter API GET /tasks (liste)	Nada	3h	À faire
US-04	UI affichage liste + état vide	Houda	3h	À faire
US-05	Vérifier persistance (save + reload depuis MySQL)	Nada	3h	À faire
US-05	Gestion erreurs basiques (API/BD) + messages simples	Ouissal	2h	À faire
US-05	Test manuel end-to-end (ajout → refresh → liste)	Ouissal	2h	À faire

3.3 Sprint Backlog détaillé — Sprint 2

US	Sous-tâche technique	Responsable	Estimation (h)	Statut
US-06	Implémenter API PUT /tasks/{id} (modifier)	Nada	4h	À faire
US-06	UI modification (mode édition / modal)	Houda	4h	À faire
US-07	Implémenter API DELETE /tasks/{id}	Nada	3h	À faire
US-07	UI suppression + confirmation	Houda	2h	À faire
US-08	Implémenter API PATCH /tasks/{id}/done (toggle)	Nada	3h	À faire
US-08	UI “terminée” (style barré/badge)	Houda	2h	À faire
US-09	Messages d’erreur front (champ requis, etc.)	Houda	2h	À faire
US-09	Harmoniser erreurs back (codes HTTP + message)	Ouissal	2h	À faire
US-10	Ajouter tests unitaires (service/controller)	Ouissal	4h	À faire
US-10	Vérifier pipeline CI + corrections jusqu’à “green”	Ouissal	2h	À faire
US-11	Préparer configuration recette (variables env, BD)	Ouissal	3h	À faire
US-11	Déployer en recette (plateforme choisie)	Ouissal + Nada	4h	À faire

US-11	Smoke tests en recette (CRUD complet)	Ouissal	2h	À faire
-------	---------------------------------------	---------	----	---------

Partie 4 — Simulation des cérémonies Scrum (Sprint 1)

Contexte Sprint 1

- **US du Sprint 1** : US-01, US-02, US-03, US-04, US-05 (16 points)
- **Sous-tâche importante** : “Créer Entity/DTO Task (title, done, dates)” est **dans US-03** car elle sert directement à “Ajouter une tâche”.

4.1 Sprint Planning (Sprint 1)

- **Durée** : 1h
- **Participants** : Houda (PO), Ouissal (SM), Nada (Dev)

Sprint Goal :

Livrer un premier MVP : page “Mes tâches” + ajout + affichage + sauvegarde en base.

Stories sélectionnées : US-01, US-02, US-03, US-04, US-05 (priorité Must)

Décision importante :

L'équipe décide de commencer par :

1. **US-01** (socle) : repo + Spring Boot + MySQL + CI
2. **US-03** : **Créer Entity/DTO Task** dès le début, car c'est la base pour l'API POST /tasks
3. **US-02/US-04** : UI + affichage liste
4. **US-05** : vérifier la persistance (refresh)

Risques identifiés :

- Front/Back (CORS, JSON) → tester d'abord l'API via Postman.
- Configuration MySQL → documenter la config dans README.

4.2 Exemple de Daily Scrum — Jour 2 du Sprint 1

Membre	Qu'ai-je fait hier? J'ai créé le repo et la structure du projet.	Que vais-je faire aujourd'hui ? J'ai initialisé Spring Boot et connecté MySQL.	Blocages ?
Ouissal (SM)	J'ai créé le repo et la structure du projet.	Mettre en place la CI (GitHub Actions).	Oui, mais j'ai résolu le problème.
Nada (Dev)	J'ai initialisé Spring Boot et connecté MySQL.	Créer Entity/DTO Task (title, done, dates) + commencer POST /tasks.	Un peu.
Houda (PO)	J'ai commencé la page “Mes tâches” (HTML/CSS).	Ajouter le formulaire et préparer l'affichage de la liste.	Non.

4.3 Sprint Review — Fin Sprint 1

- **Durée** : 40 min
- **Fonctionnalités démontrées** :
 - o Socle technique OK (Spring Boot + MySQL + repo)
 - o Page “Mes tâches” affichée
 - o Ajout d’une tâche fonctionnel (titre obligatoire)
 - o Liste des tâches affichée
- **Ce qui a bien aidé** :

Le fait d’avoir fait tôt la tâche “**Créer Entity/DTO Task**” a accéléré le développement de l’API et la persistance.
- **Non terminé / à améliorer** :
 - o Persistance parfaite + gestion d’erreurs plus propre (finir dans Sprint 2 si besoin)
- **Vélocité réelle** :
 - o **13 points terminés sur 16** (on reporte une partie de US-05 au Sprint 2)

4.4 Sprint Rétrospective — Start / Stop / Continue

Start	Stop	Continue
Tester l’API avec Postman dès qu’un endpoint est prêt	Attendre la fin pour intégrer front/back	Daily court (10–15min)
Faire une PR avant merge (petite revue)	Tout faire sur une seule branche	Découper en sous-tâches (2–4h)
Noter les blocages dans Jira	Sous-estimer l’intégration UI/API	Commits réguliers

Partie 5 — Definition of Ready (DoR) et Definition of Done (DoD)

5.1 Definition of Ready (DoR) — “Prêt à entrer dans un Sprint”

Une User Story peut être prise dans **Sprint 1 ou Sprint 2** seulement si elle respecte les critères suivants :

#	Critère DoR (objectif et vérifiable)	Dimension
1	La User Story est écrite au format : “ En tant que..., je veux..., afin de... ”	Compréhension
2	Elle contient au moins 2 critères d’acceptation clairs et testables	Qualité fonctionnelle
3	Elle est estimée en Story Points (1,2,3,5,8...) et l’équipe est d’accord sur l’estimation	Estimation
4	La priorité MoSCoW est définie (Must/Should/Could/Won’t)	Priorisation

5	Les dépendances sont identifiées (ex : US-01 socle avant API tasks ; format JSON connu pour UI)	Dépendances
6	Les sous-tâches principales du Sprint Backlog sont identifiées (ex : pour US-03 inclure “Créer Entity/DTO Task...”)	Préparation technique

5.2 Definition of Done (DoD) — “Terminé” (contrat qualité)

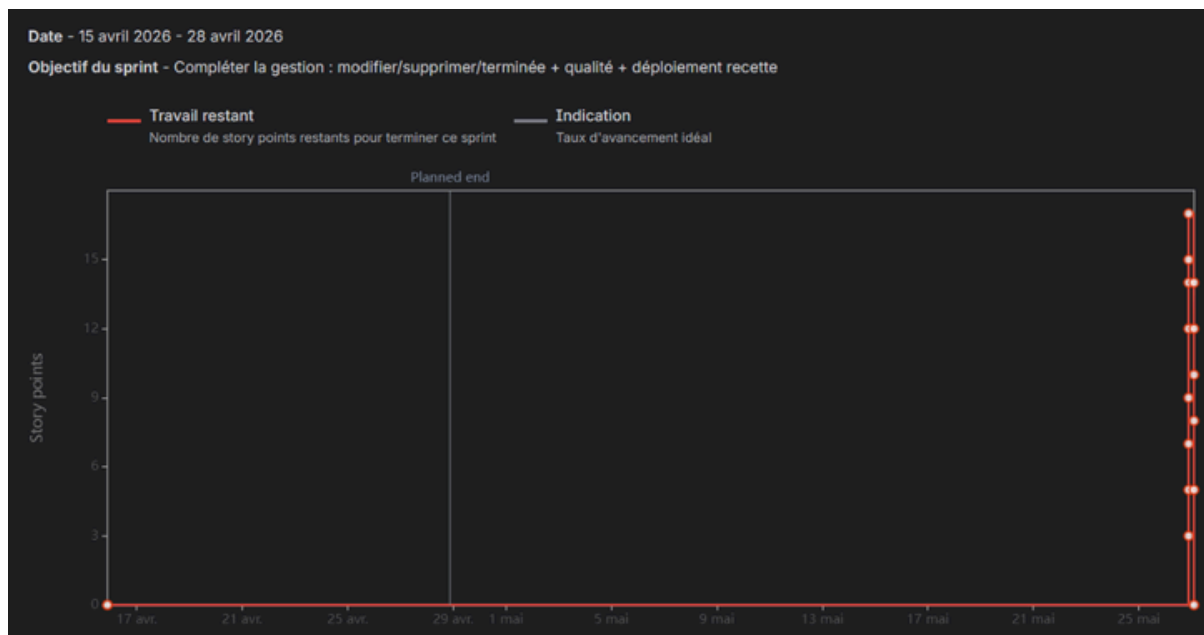
Une User Story est considérée comme **Done** seulement si toutes les conditions ci-dessous sont vraies :

#	Critère DoD (objectif et vérifiable)	Dimension
1	Tous les critères d’acceptation de la User Story sont respectés	Fonctionnel
2	Le code est poussé sur GitHub et intégré via une Pull Request (revue par au moins 1 membre)	Qualité code
3	La CI (GitHub Actions) est verte après merge (build OK)	Intégration / CI
4	Les tests nécessaires sont réalisés : au minimum test manuel + (si applicable) tests unitaires passent	Tests
5	Les endpoints concernés (ex: POST/GET/PUT/DELETE /tasks) sont testés (Postman ou équivalent) et ne renvoient pas d’erreurs	Tests API
6	La fonctionnalité est stable : pas de bug bloquant lors d’un scénario simple (ex : ajouter → refresh → liste)	Qualité / stabilité
7	La documentation minimale est à jour (README : comment lancer le projet, config MySQL, etc.) si la story modifie la configuration	Documentation
8	Si la story concerne le déploiement (ex: US11), la fonctionnalité est vérifiée en recette (URL accessible + smoke test)	DEPLOIEMENT

Partie 6 — Suivi et indicateurs

6.1 Burndown Chart du Sprint 2

Le graphique ci-dessous est le **Burndown Chart réel** extrait de Jira pour le **Sprint 2** (15 avril → 28 avril).



Analyse du graphique :

- Le travail restant est resté à **0** pendant presque tout le sprint.
- La majorité des Story Points a été terminée **le dernier jour** du sprint (chute verticale).
- Cela montre que les tâches ont été réalisées de façon **très concentrée** sur une seule journée, et non de manière progressive.

6.2 Outils de gestion de projet

Outil utilisé : Jira (Scrum Board)

Justification : Jira permet de gérer le Product Backlog, planifier les sprints, suivre les sous-tâches et générer automatiquement le Burndown Chart.

Lien Jira :

- [<https://dailyplanz.atlassian.net/jira/software/projects/US/boards/1/backlog?atlOrigin=eyJpIjoiY2I0MjQ3ZWE3NzljNDI3ZmE3ZWRIInJkwMWFhMmMxYTgiLCJwIjoiaj9>]

6.3 Analyse de 2 risques liés à Scrum

#	Risque identifié	Mesure de mitigation
1	Travail concentré sur une seule journée (comme montré dans le Burndown Chart) → risque de tests insuffisants et de qualité moindre	Réserver du temps dans le sprint pour les tests et les corrections, et respecter la Definition of Done avant de marquer une tâche comme terminée.
2	Mauvaise répartition du travail dans le temps → surcharge à la fin du sprint	Découper les tâches en sous-tâches plus petites et mettre à jour le Burndown Chart régulièrement tout au long du sprint.

Remarque:

*Nous avons d'abord développé le site sans mettre les tâches en statut « terminée ».
Ensuite, pour préparer les livrables, nous avons mis toutes les tâches en « terminée »
le même jour.*